a css grid design is having a side bar on the right of 1 unit (1/6 of total page width) and another element on the left that has the remaining width (5/6 of total width). compare 1) divide the width of the left element to three parts using grid-template-areas
2)using a main grid of two parts and the divide the left part into three using another sub-grid?

Here is the short answer: **Option 2 (Nested Grid / Subgrid)** is generally cleaner and far more flexible for modular code, while **Option 1 (Single-Level Grid)** works best if you need elements inside the left container to align pixel-for-pixel with elements in the right sidebar across vertical rows.

Here is how both approaches work, their math, and how to choose between them.

Method 1: Single Grid using `grid-template-areas`

In a single top-level grid, the parent defines columns for both the left section's internal columns and the right sidebar simultaneously.

To make the left side total 5/6th of the page (divided into 3 equal parts) and the right sidebar 1/6th, we multiply the units so every column is an integer:

Left side: $3 \times 5\text{fr} = 15\text{fr}$ (split into three 5fr columns)

Right side: 3fr (which equals $3/18 = 1/6$ of the total)

CSS

```
.page-container {
  display: grid;
```

```
  grid-template-columns: 5fr 5fr 5fr 3fr;
  grid-template-areas:
    "left-1 left-2 left-3 sidebar"
    "left-main left-main left-main sidebar";
  gap: 1rem;
}

.item-a { grid-area: left-1; }
.item-b { grid-area: left-2; }
.item-c { grid-area: left-3; }
.sidebar { grid-area: sidebar; }
```

Pros

Row Alignment across sections: If items in `left-1` and the `sidebar` need to share horizontal row heights seamlessly, a single top-level grid forces strict row alignment across the entire page.

Cons

Coupling: The main layout container must explicitly know about the internal child layout of the left section.

Complex Math with Gaps: Adding a `gap` to the parent grid alters the strict 5:1 ratio calculation slightly.

Method 2: Two-Part Main Grid + Subgrid / Nested Grid

Here, the main grid only handles the top-level $5:1$ split. The left element handles its own 3-column subdivision independently (either via standard nested grid or CSS `subgrid`).

CSS

```
/* 1. Top-Level Page Grid */
.page-container {
  display: grid;
  grid-template-columns: 5fr 1fr;
  gap: 1.5rem;
}
```

```
/* 2. Left Element (Nested Grid) */
.left-content {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}

/* Optional: Subgrid Variation (if left-content items need main grid alignment
.left-content-subgrid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  /* Inherits row tracks from .page-container if needed */
  grid-template-rows: subgrid;
}
```

HTML

```
<div class="page-container">
  <main class="left-content">
    <div>Part 1</div>
    <div>Part 2</div>
    <div>Part 3</div>
  </main>
  <aside class="sidebar">Sidebar (1/6)</aside>
</div>
```

Pros

Encapsulated & Modular: The left component manages its own internal structure. You can reuse `.left-content` elsewhere without needing a 4-column parent grid.

Simpler Ratio Math: The main grid cleanly uses `5fr 1fr` without needing common multiples like `5fr 5fr 5fr 3fr`.

Independent Gaps: You can set a wide gap between the main area and sidebar (`gap: 2rem`), while keeping smaller gaps inside the left grid (`gap: 0.75rem`).

Cons

- **Independent Rows:** Without CSS `subgrid` on rows, row heights inside `.left-content` will not automatically sync with row heights inside `.sidebar`.

Comparison Summary

Feature	Single Grid (`grid-template-areas`)	Nested Grid / Subgrid
Separation of Concerns	Low (Parent knows child details)	**High** (Clean encapsulation)
Ratio Math Cleanliness	Scaled fractions (`5fr 5fr 5fr 3fr`)	**Direct** (`5fr 1fr`)
Flexibility / Maintenance	Harder to refactor child items	**Easy** to swap out left-side layouts
Cross-Sidebar Row Sync	**Native** across all columns	Requires CSS `subgrid`

Recommendation

Use **Method 2 (Nested Grid)**. It yields significantly maintainable, component-driven CSS. Reserve **Method 1** only for cases where elements in the left columns must strictly lock horizontal row alignment with elements in the right sidebar.